

Context7 Architecture at a Glance

1. Surface Comparison

Surface	Primary Purpose	Main Users	API Path Used	Strengths	Tradeoffs
MCP Server (@upstash/context7-mcp)	Native tool calls from MCP-capable AI clients	IDE agents (Cursor, Claude Code, VS Code MCP, etc.)	GET /api/v2/libs/search, GET /api/v2/context	Standardized tool interface, HTTP and stdio transports, session and auth support	MCP client compatibility variance, operational complexity in HTTP mode
CLI (ctx7)	Setup/remove automation + direct docs lookup in terminal	Developers and platform/tooling teams	Same core docs endpoints + setup/auth endpoints	Fast onboarding, multi-client config automation, practical fallback when MCP unavailable	Installer logic must handle many client config formats
TypeScript SDK (@upstash/context7-sdk)	Programmatic REST integration in apps/services	App/backend engineers	Same core docs endpoints	Minimal typed client, clean embedding in code, retry/backoff support	You must implement your own orchestration/prompt workflow
Tools AI SDK (@upstash/context7-tools-ai-sdk)	Vercel AI SDK-native tool wrappers + preconfigured agent	AI product teams	Via TS SDK -> same endpoints	Ready-made resolve-then-query workflow, faster agent integration	Depends on AI SDK conventions and prompt behavior
pi Extension (@upstash/context7-pi)	pi.dev-native tools and slash command integration	pi.dev users	Same core docs endpoints	Parity with MCP tool semantics, low-friction integration	Narrow ecosystem scope compared with generic MCP

2. Operational Concerns by Surface

Concern	MCP Server	CLI	SDK	Tools AI SDK
Authentication	API key headers; OAuth endpoint; JWT validation in HTTP mode	Login/API key setup flows; writes credentials into client configs	API key in constructor/env	Inherits SDK auth model
Transport	stdio and Streamable HTTP	Local command execution	HTTP client only	AI tool-call layer over SDK

Concern	MCP Server	CLI	SDK	Tools AI SDK
Session State	Redis-backed sessions (HTTP mode), per-process session for stdio	Not session-oriented	Not session-oriented	Not session-oriented
Config Management	Managed by MCP client	Strong automation for JSON/JSONC/TOML config writing and patching	App-managed	App/agent-managed
Failure Handling	Error normalization + arg-aliasing for LLM schema drift	User-facing command errors + guided remediation	Throws typed errors	Tool-returned error messages with workflow hints

3. What Is Open vs Private

Layer	Visibility in Repository	Notes
MCP server implementation	Open	Full server/tool code is in <code>packages/mcp</code>
CLI installer and docs commands	Open	Full setup/remove/docs/auth flows are in <code>packages/cli</code>
SDK and AI wrappers	Open	Full source in <code>packages/sdk</code> and <code>packages/tools-ai-sdk</code>
Parsing/crawling/indexing backend engines	Private	Explicitly documented as not included in this repo
Hosted ranking/retrieval internals	Private (API contract visible)	Behavior inferred from OpenAPI/docs and client integrations

4. Recommended Usage by Team Type

Team Type	Recommended Primary Surface	Why	Optional Secondary Surface
Individual developer using MCP-capable IDE	CLI setup + MCP server	Fast setup and native in-editor tool calls	CLI docs commands for ad hoc terminal lookups
AI app team building custom workflows	TypeScript SDK	Full code-level control and easier service integration	Tools AI SDK for rapid prototyping
Agent platform team standardizing multiple clients	CLI + MCP	Best support for cross-client setup, config, and policy rollouts	Plugin artifacts in <code>plugins/</code> as templates
Enterprise/on-prem evaluation team	MCP interface + API contracts	Preserves client compatibility while allowing infra control	On-prem docs/deployment guides for private environments

5. Integration Decision Matrix

If your priority is...	Choose...
Fastest IDE activation with minimal coding	<code>ctx7 setup</code> + MCP
Building productized backend/API features	TypeScript SDK
Agentic workflows with Vercel AI SDK	Tools AI SDK
Cross-client governance and repeatable rollout	CLI automation + rules/skills templates
Single-command manual docs retrieval in shell	CLI (<code>ctx7 library</code> , <code>ctx7 docs</code>)

6. One-Paragraph Executive Summary

Context7's open-source architecture is a delivery and integration plane centered on a stable two-step retrieval contract: resolve library ID, then query docs context. MCP, CLI, SDK, and AI tool wrappers all converge on the same hosted API endpoints, while private backend services handle parsing, crawling, and indexing outside this repository. The design strongly favors broad client compatibility, predictable orchestration, and rapid onboarding across developer tooling ecosystems.